UNIVERSITÀ DI PAVIA

Cryptocurrency Factor Forecasting & Structural Validation

Financial Data Science

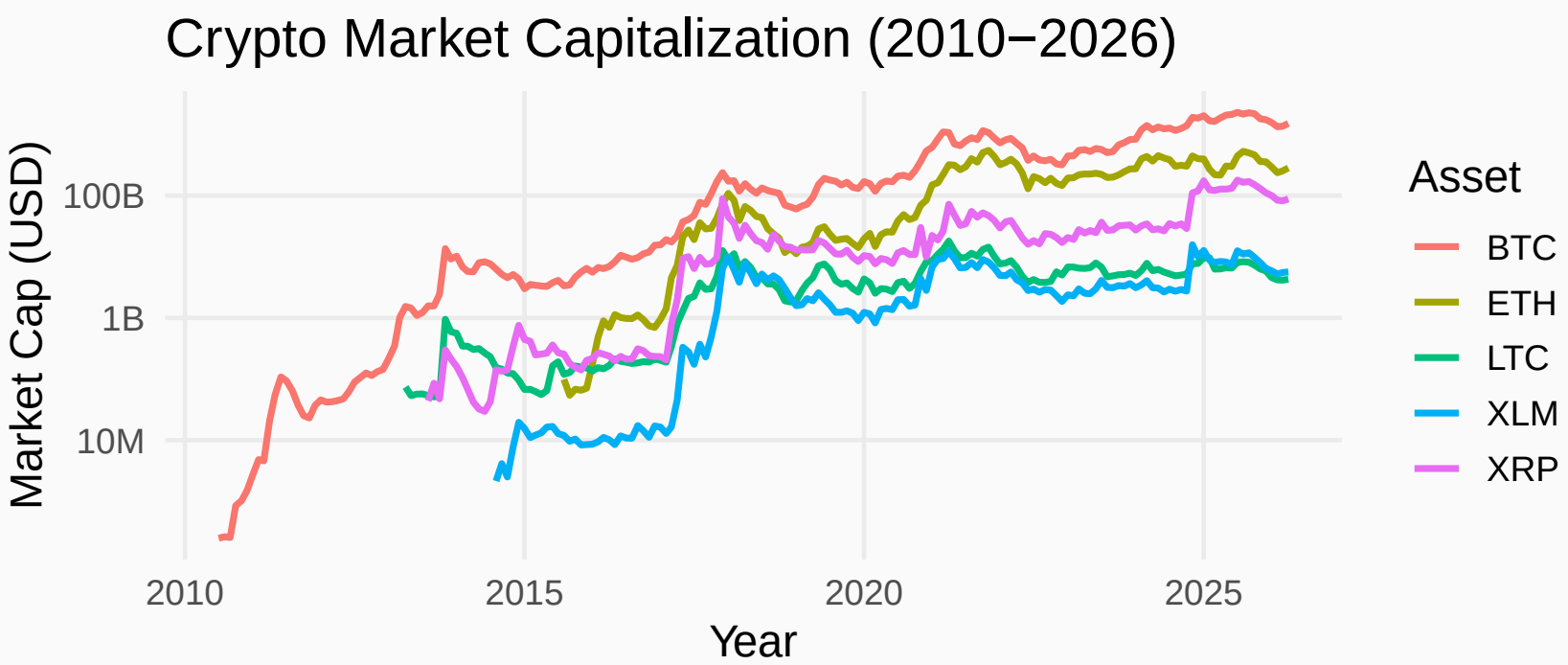
Claudio Guarrasi
claudio.guarrasi01@universitadipavia.it

April 25, 2026

Department of Electrical, Computer and Biomedical Engineering
University of Pavia

Motivation

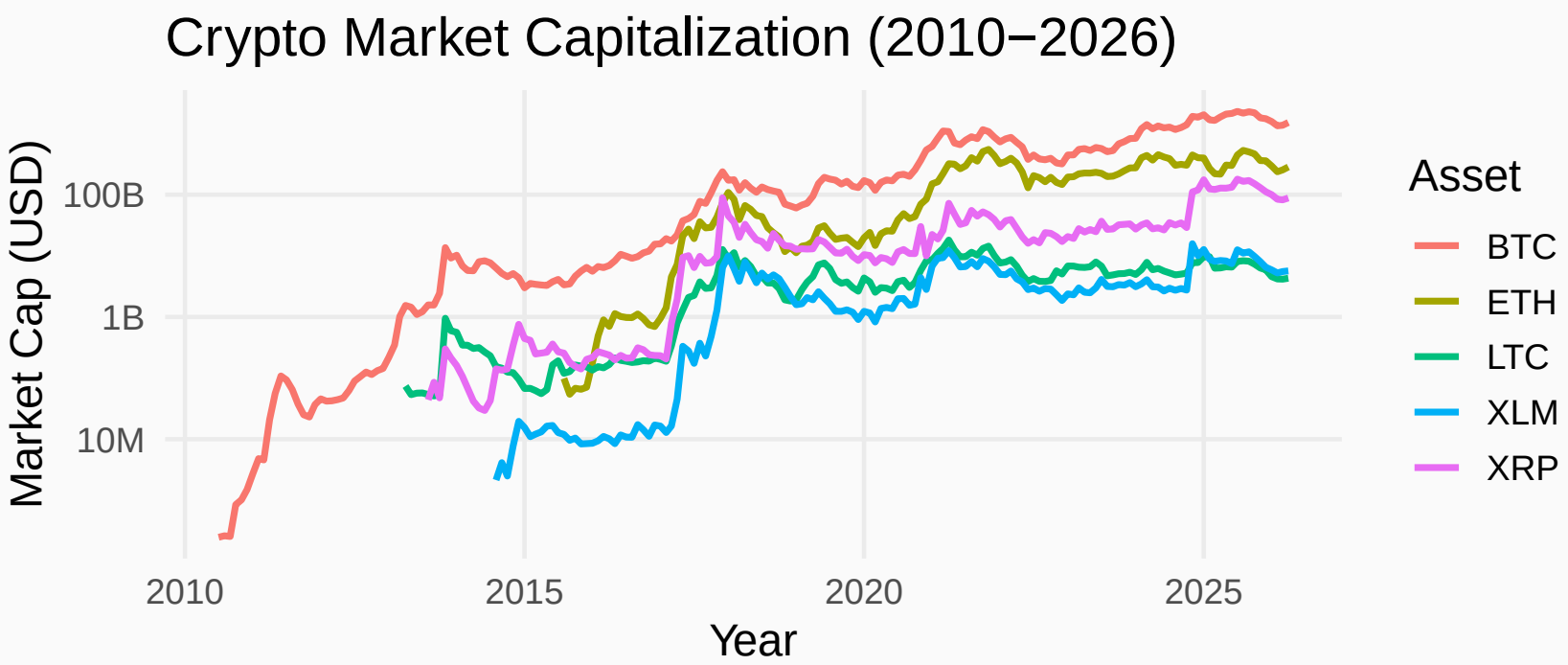
Cryptos Market Has Become Huge



- Market capitalization of cryptos has reached approximately \$2.57 trillion[1], comparable to the GDP of Italy[2].
- The general population buys these assets with the hope of making more money.

[1] `https://coinmarketcap.com/charts/` (Accessed on 18/04/26)
[2] IMF World Economic Outlook, Oct 2025

A Very Volatile Market



- Extreme events happen rather frequently, see figure.
- Buying these assets exposes to a risk.
- Developing a model to capture the common factors driving these movements.

Factor Model

Dataset Structure

- **Dimensions:** $N = 5$ assets across $T = 1,369$ daily observations.

Date (T)	**ETH**	**BTC**	**XRP**	**LTC**	**XLM**
2016-01-01	\$0.95	\$434.33	\$0.006	\$3.51	\$0.002
...	...	...	...	...	...
2019-09-30	\$179.87	\$8293.87	\$0.26	\$56.06	\$0.06

Structure of the X matrix ($T \times N$).

- Only asset prices are present.
- Common factors are unobservable.

Factor Model

- Enables estimation in the presence of **unobserved predictors**.
- Assumes that $X_{i,t}$ is driven by a small, **unknown number** (k) of latent common factors:

$$X_{i,t} = \phi_i' F_t + u_{i,t}, \quad \begin{cases} i = 1, \ldots, N \\ t = 1, \ldots, T \end{cases}$$

Model Components

F_t **Common Factors:** $k \times 1$ vector of market drivers.

ϕ_i **Factor Loadings:** $k \times 1$ vector describing the impact of common factors on asset i.

$u_{i,t}$ **Idiosyncratic Component:** Residual asset-specific noise.

Main Assumptions

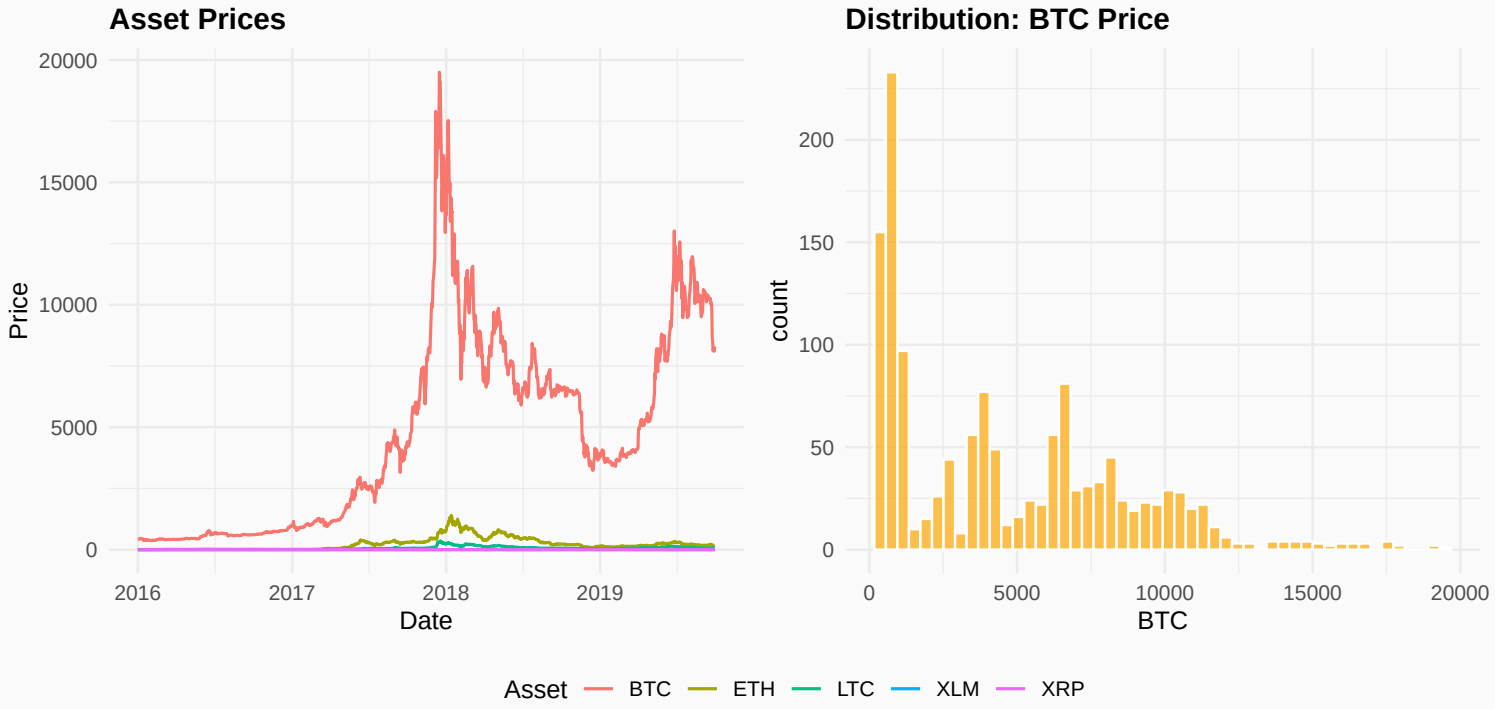
To ensure the mathematical consistency of the factor model, we assume:

- **Asymptotics:** $\min\{N, T\} \to \infty$ for the consistent estimation of both the factor space and the true number of common factors k.
- **Stationarity:** $X_{i,t}$ is a stationary process, ensuring stable moment properties over time.
- **Moment Conditions:** $E|u_{i,t}|^8 < \infty$. High-order moments are required for the asymptotic convergence of the sample covariance matrices.

Covariance Structure and Eigenvalues
Let $\Sigma_F = E[F_t F_t'] > 0$. Under the existence of the 4th moment, the first k eigenvalues of $\hat{\Sigma}_X$ diverge:

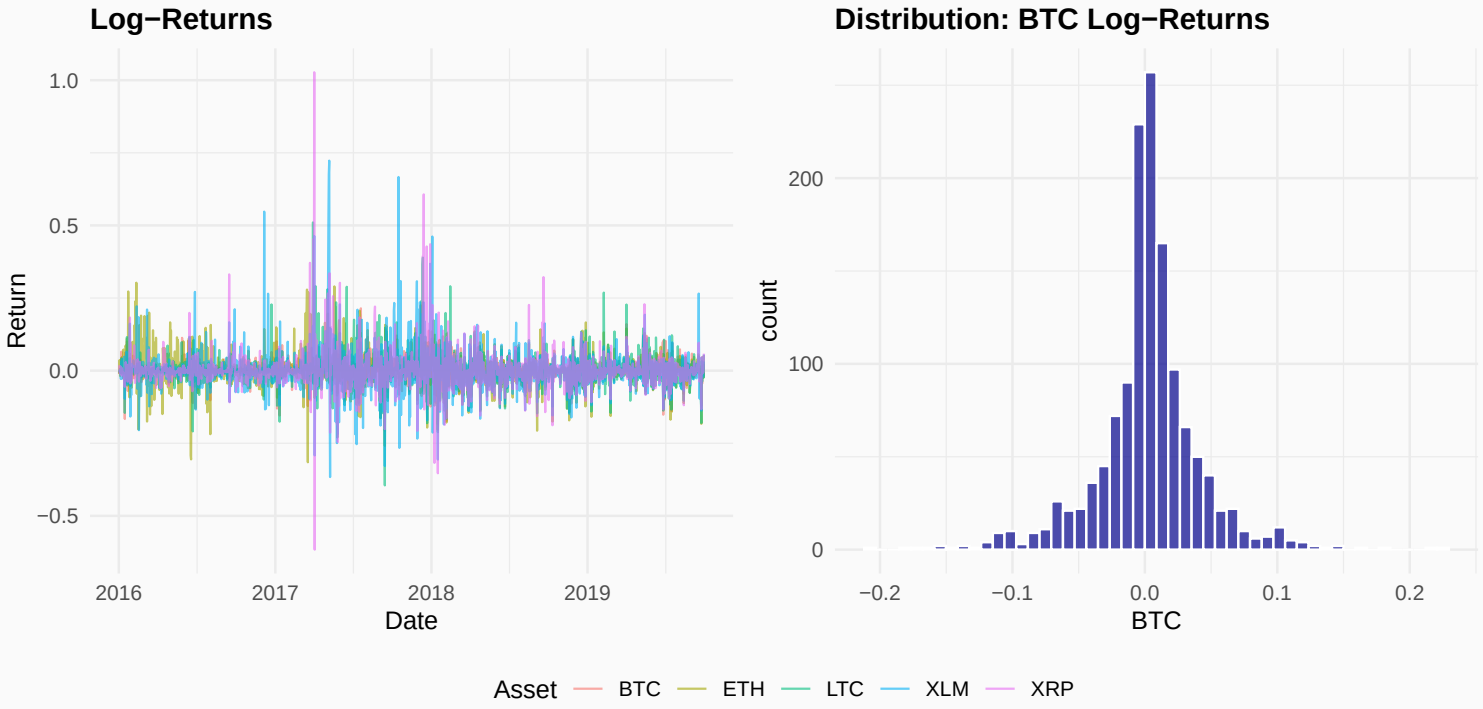
$$\lambda_j(\hat{\Sigma}_X) \geq c_0 N, \quad \text{for } j = 1, \ldots, k$$

Assumptions Are Satisfied?



- $N = 5$ is small.
- Prices exhibit a strong upward stochastic trend (non-constant mean).
- The BTC histogram shows extreme right-skewness.
- Assets are on different scales.

Data Transformation: Log-Returns



- $y_{i,t} = \ln\left(\frac{x_{i,t}}{x_{i,t-1}}\right) = \ln(x_{i,t}) - \ln(x_{i,t-1})$.
- This transformation centers the distribution around zero, stabilizes the variance and reduces the impact of outliers.
- Now assets with vastly different nominal prices are brought to a common scale.

Initial Approach: Estimating the Number of Common Factors

To determine the optimal number of factors $\hat{k}$, we employ the information criteria proposed by **Bai and Ng (2002)**:

- **Criteria:** We consider IC_{p1}, IC_{p2}, and IC_{p3}, which balance the minimization of the sum of squared residuals against a penalty for model complexity.
- **Implementation in R:** Utilizing the `ICr` function from the `dfms` package, we find $\hat{k}$ such that:

$$\hat{k} = \arg \min_{0 \leq k \leq k_{max}} IC(k)$$

  where the search space k_{max} is determined by Schwert's rule of thumb.

Initial Approach: Static Factor Estimation

- **Estimation:** Common factors $\hat{F}_t$ are estimated via Principal Component Analysis (PCA) on the full sample of log-returns.
- **Dimension Reduction:** $\hat{F}_t$ corresponds to the $\hat{k}$ largest eigenvalues of the sample covariance matrix $\hat{\Sigma}_X$.

Limitations:

- Factors lack direct interpretability.
- Difficulty in assessing the model's predictive accuracy.

Transition to Stock & Watson (2002)
To evaluate the "goodness of fit," we adopt the **Stock and Watson methodology**, which enables real-time simulation and benchmarking.

Stock & Watson Methodology

Stock & Watson Methodology

- **Data Split:** The sample is divided into a **training set** (75%) and a **test set** (25%).
- **Recursive Estimation:** For each time t in the test set:
  1. Factors $\hat{F}_t$ are extracted via PCA from the training window.
  2. A linear model is estimated via OLS for each asset using $\hat{F}_t$
  3. A 1-step ahead forecast is generated and stored: $\hat{y}_{i,t+1} = \hat{\beta}_i \hat{F}_t$.
  4. The training window expands to include the observation at t.
- **Benchmarking:** An **AR(p)** model is estimated at each step:

$$y_{i,t+1} = \beta_0 + \sum_{j=1}^{p} \beta_j y_{t-j} + \epsilon_{t+1}$$

  where p is selected dynamically via **BIC**.
- **Evaluation:** Model utility is determined by comparing the **Mean Squared Error (MSE)** of the Factor Model against the AR benchmark.

Theoretical Consistency and Identifiability

The framework provides mathematical assurance regarding the consistent estimation of common factors:

- **Consistency up to Rotation:** PCA consistently estimates HF_t rather than F_t, where H is a non-singular $k \times k$ rotation matrix. The R implementation utilizes the `PCA` function from the `HDRFA` package.
- **Simplification to Sign Uncertainty:** Stock and Watson (2002) show that under our assumptions and distinct eigenvalues, the rotation H simplifies to a sign-switching matrix S.
  - Factors are essentially identified up to a **change of sign** ($\pm\hat{F}_t$).
- **Resolution via OLS:** This sign misalignment is automatically compensated for by the estimated OLS parameters in the linear regression model:

$$\hat{y}_{i,t+1} = \hat{\beta}_i' \hat{F}_t$$

Results: Factor Model vs. AR Benchmark

To evaluate performance, we calculate the **Relative Mean Squared Error** for each asset:

$$\text{RelMSE}_i = \frac{MSE_{i,\text{Factor}}}{MSE_{i,\text{AR}}}$$

A ratio < 1 indicates that the factor model provides a gain in predictive accuracy over the benchmark.

Findings:

- For the analyzed cryptocurrencies, the **Relative MSE is approximately 1** in each case. This suggests that the factor model and the AR model yield nearly equivalent forecast errors.
- For **3 out of 5 assets**, the BIC-optimal lag was $p = 0$, whereas the estimated number of common factors was $\hat{k} = 3$.
- **Interpretation:** For the majority of the sample, the best predictor is simply the **historical sample mean**.

Robustness Check: Expanding the Cross-Section (N)

To determine if the previous results were due to the small cross-section ($N = 5$), we expanded the dataset using the `crypto2` package:

- We cleaned the data inspired by the methodology of **Bianchi and Babiak (2022)**, filtering for liquidity and market capitalization to ensure a high-quality data pool.
- We increased the number of assets ($N = 29$) to test if a larger dataset would improve the prediction accuracy.
- **Findings:**
  - The **Relative MSE results remained analogous** to the small-sample case (RelMSE ≈ 1).
  - Expanding the number of assets did not yield superior predictive performance over the AR(0) benchmark.

The Fundamental Question
Does a factor structure actually exist in the data (i.e. $k > 0$)?

Testing for Factor Strength: The Trapani (2018) Test

To determine if the factors are "strong" or merely noise, we test the behavior of the eigenvalues $\lambda^{(p)}$ for $p = 1, \ldots, \hat{k}$ as $N \to \infty$:

- **Hypotheses:**
  - $H_0 : \lambda^{(p)} = O(N)$ (Diverging eigenvalue; strong factor).
  - $H_A : \lambda^{(p)} = O(1)$ (Bounded eigenvalue; noise).
- **Test Statistic:** Trapani constructs a statistic $\Theta^{(p)}$ that, under H_0, follows a **chi-squared distribution**:

$$\Theta^{(p)} \sim \chi^2(1)$$

- **Empirical Results:**
  - **Small Dataset ($N = 5$):** We reject H_0 for the largest eigenvalue ($p = 1$). Consequently, $\hat{k} = 0$, indicating the absence of a common factor structure.
  - **Expanded Dataset ($N = 29$):** We reject H_0 for the second largest eigenvalues ($\hat{k} = 1$).

Conclusions

This work evaluated the presence and predictive utility of common factors in the cryptocurrency market using the **Stock & Watson (2002)** framework and **Trapani (2018)** test.

- **Predictive Performance:** In both small and large datasets, the factor model yielded a **Relative MSE** $\approx$ **1**, failing to outperform a simple AR(0) benchmark.
- **Identification Sensitivity:** The **Trapani (2018)** test reveals that factor identification is highly dependent on the cross-sectional dimension (N).
- **Sample-Size Dependency:** No consistent factor structure was identifiable for $N = 5$ ($\hat{k} = 0$), whereas a statistically diverging signal begins to emerge for $N = 29$ ($\hat{k} = 1$).

Future Work

The findings of this research open several avenues for extending the analysis of cryptocurrency co-movements and predictive modeling:

- **Time-Varying Loadings:** Implementing **Dynamic Factor Models** where factor loadings $\phi_{i,t}$ can evolve, accounting for the rapidly changing nature of the cryptocurrency market.
- **Non-Linear Factor Models:** Investigating **Machine Learning** architectures (e.g., Autoencoders or Recurrent Neural Networks) to capture non-linear dependencies that PCA-based linear models may miss.

References

Bibliography i

J. Bai and S. Ng.
Determining the Number of Factors in Approximate Factor Models.
Econometrica, 70(1):191–221, 2002.

D. Bianchi and M. Babiak.
A factor model for cryptocurrency returns.
CERGE-EI Working Paper Series 710, 2021.

J. H. Stock and M. W. Watson.
Forecasting Using Principal Components From a Large Number of Predictors.
Journal of the American Statistical Association, 97(460):1167–1179, 2002.

Bibliography ii

J. H. Stock and M. W. Watson.
Macroeconomic Forecasting Using Diffusion Indexes.
Journal of Business & Economic Statistics, 20(2):147–162, 2002.

L. Trapani.
A Randomized Sequential Procedure to Determine the Number of Factors.
Journal of the American Statistical Association, 113(523):1341–1349, 2018.

Appendix: Source Code

Main Script i

```
# Clear the environment
rm(list=ls())

# Setting as working directory the folder of
    this script
setwd("~/Desktop/financial_data_science/cryptos-
    factor-model-estimation")
# Loading the real-time forecasting function,
    implemented according Stock and Watson
    methodology,
# which also integrates the Trapani test to
    collect evidence about the presence of a
    common factor structure in the data.
source("real-time_forecasting.R")
```

Main Script ii

```
#source("real-time_forecasting2.R")

# Install packages
# install.packages("ggplot2",dependencies=TRUE)
# install.packages("readxl",dependencies=TRUE)
# install.packages("corrplot",dependencies=TRUE)
# install.packages("dfms", depedencies=TRUE) #
   To estimate number of factor in a static
   factor model under
# information criteria. (Static factor model are
    a subset of dynamical factor model).
# It handles dataset with missing data
# install.packages("xts", dependencies=TRUE) #
   General package to study time-series dataset
```

Main Script iii

```
# install.packages("HDRFA", dependencies=TRUE) #
    it contains functions to estimate k
# with PCA and more, but also techinique to
   handle data in which outliers are frequent (
   robust PCA)
#install.packages("crypto2", dependencies=TRUE)
#install.packages("readr", dependencies=TRUE)

# Call libraries
library(readxl) # To use 'read_excel'
library(dfms) # To use 'ICr'
library(HDRFA) # To use 'PCA'
library(crypto2) #To use 'crypto_list', 'crypto_
   history'
```

Main Script iv

```
library(dplyr) #To use '%>%'
library(tidyr) #To use 'pivot_wider'
library(readr) # To use 'read_csv', 'write_csv'
    ...
library(zoo) # To use 'na.locf'

# Import dataset
crypto_dataset<-read_excel("dataset1_cryptos.
    xlsx")



# Explore data
str(crypto_dataset) # Display the internal str
    ucture of an R object.
```

Main Script v

```
summary(crypto_dataset$ETH) # Display basic
    summary statistics
summary(crypto_dataset$BTC) # This is enough to
    see that the sample distributions
summary(crypto_dataset$XRP) # of crypto assets
    are not symmetric.
summary(crypto_dataset$LTC) # They are skewed to
     the right.
summary(crypto_dataset$XLM) # Data are not "well
     behaved". Variance is not meaningful.

hist(crypto_dataset$ETH) # Histograms give us an
     idea of the shape of the data
```

Main Script vi

```
hist(crypto_dataset$BTC) # distributions. In our
    case we see long tails of values
hist(crypto_dataset$XRP) # to the right.
   Confirming our guess about the absence
hist(crypto_dataset$LTC) # symmetry.
hist(crypto_dataset$XLM)

# Boxplots provide us an equivalent point of
   view about the patterns in our data
boxplot(crypto_dataset$ETH, main="Boxplot␣of␣ETH
   ␣Asset␣Price", ylab="Price␣($)")
quantile(crypto_dataset$ETH)
```

Main Script vii

```
boxplot(crypto_dataset$BTC, main="Boxplot␣of␣BTC
    ␣Asset␣Price", ylab="Price␣($)")
quantile(crypto_dataset$BTC)

boxplot(crypto_dataset$XRP, main="Boxplot␣of␣XRP
    ␣Asset␣Price", ylab="Price␣($)")
quantile(crypto_dataset$XRP)

boxplot(crypto_dataset$LTC, main="Boxplot␣of␣LTC
    ␣Asset␣Price", ylab="Price␣($)")
quantile(crypto_dataset$LTC)

boxplot(crypto_dataset$XLM, main="Boxplot␣of␣XLM
    ␣Asset␣Price", ylab="Price␣($)")
```

Main Script viii

```
quantile(crypto_dataset$XLM)




cor(crypto_dataset[-1]) # Is there collinearity?
    In the case of crypyto assets, yes.
                            # Are cryptos driven by
                                common features?

# Data pre-processing

crypto_dataset<-crypto_dataset[-1] # Removing
   the 'date' column
```

Main Script ix

```
log_crypto_dataset<-log(crypto_dataset) # The
    natural logarithm of our data

log_ret_crypto_dataset<-as.data.frame(diff(as.
    matrix(log_crypto_dataset))) # The first
    difference between consecutive data
    observation

summary(log_ret_crypto_dataset)
cor(log_ret_crypto_dataset)

# With the previous statements we obtain log-
    returns. The statistical properties
```

Main Script x

```
# of log-returns are more tractable and comply
   with the hyphoteses we make
# when applying factor models. Now the
   distributions are more symmetric and
# the weight of outliers is diminished.

# Since we suspect that cryptos share common
   factors we estimate a foctor model
# in order to try to predict future log-returns.
# We are going to compare it with an
   autoregressive (AR) linear model in order to
   see
# if it is capable of capturing more than simply
    the previous history.
```

Main Script xi

```
results <- run_stock_watson_forecast(log_ret_
    crypto_dataset)
print(results$Relative_MSE)
print(results$AR_parameter_history)
#print(results$Trapani_factor_number_history)

# A relative MSE greater than 1 means that the
    factor model performed poorer
# than the AR model, whereas a relative MSE
    smaller than 1 means that the
# factor model was better.
# In our case choosing the factor model seems to
     make no difference with respect
```

Main Script xii

```
# of choosing an AR model. Furthermore for 3
   assets over 5
# the number of AR parameters is 0, meaning that
    is more convenient to predict
# the future by simply predict the sample mean.

# Now we try to see if the reason our factor
   model acted poorly was due to the
# small dataset, specifically to the the fact
   that the number of unit N was very
# small compared to the number of observation.

# To retrieve historical prices regarding more
   cryptos we use the package "crypto2"
```

Main Script xiii

```
#all_coins <- crypto_list(only_active = TRUE) #
    We obtain a list of all coins currently
    tradeable

#top_100_coins <- all_coins[1:100, ] # We retain
     just the first 100 coins

# Now we download all what we need (specifically
     the prices along
# the same daily period of the previous dataset)
     and we treat the data acquired
# to obtain a dataset with the same structure of
     the smaller one.
```

```
#raw_data <- crypto_history(coin_list = top_100_
    coins,
                                #start_date =
                                    "2016-01-01",
                                #end_date =
                                    "2019-09-30")

#write_csv(raw_data, "crypto_raw_historical_100.
    csv") # We store the data locally
```

```
#raw_crypto_data_100<-read_csv("crypto_raw_
    historical_100.csv")
```

Main Script xvi

```
# We clean the raw data inspired by Bianchi
   Daniele and Mykola Babiak paper:
"A factor model for cryptocurrency returns."

# We keep only the coins whose trading volume is
    greater than $100 million
# because we are interested in considering coins
    which are treaded a lot.
# We keep only the coins which have at max the
   25% of missing data
# We filter-out stable coins since they are
   linked to actual coins and then they
```

Main Script xvii

```
# do not have the typical volatility of a crypto
    coin (we would like to model
# that volatility).
market_cap_floor <- 100000000  #
missing_threshold <- 0.25      # 25% Max Missing
    Data
stablecoins <- c("USDT", "USDC", "TUSD", "PAX",
   "DAI", "USDS", "GUSD")

filtered_data <- raw_crypto_data_100 %>%
  filter(!(symbol %in% stablecoins))

valid_symbols <- filtered_data %>%
  group_by(symbol) %>%
```

Main Script xviii

```
  summarize(max_cap = max(market_cap, na.rm =
     TRUE)) %>%
  filter(max_cap >= market_cap_floor) %>%
  pull(symbol)


filtered_data <- filtered_data %>%
  filter(symbol %in% valid_symbols)
#str(filtered_data)
wide_prices <- filtered_data %>%
  select(time_open, symbol, close) %>%
  rename(Date = time_open) %>%
  distinct(Date, symbol, .keep_all = TRUE) %>%
```

```
  pivot_wider(names_from = symbol, values_from =
      close) %>%
  arrange(Date)

n_obs <- nrow(wide_prices)
keep_cols <- colSums(is.na(wide_prices)) / n_obs
    <= missing_threshold
final_prices <- wide_prices[, keep_cols]
#colSums(is.na(log_final_prices))
# is.na(wide_prices) is a matrix which has TRUE
   (1) where there is a missing value in the
   original matrix
```

Main Script xx

```
# colSums(is.na(wide_prices)) is a 1D vector so
    that each elements is the number of data
    missing in the
# corresponding column in the original matrix,
    this number then becomes a percentage,
    finally if it is
# smaller than our threshold then we obtain a 1
    (TRUE) that is used to mantain the column (
    coin) in
# final matrix.


write_csv(final_prices, "crypto_simplified_final
    _prices_100.csv")
```

Main Script xxi

```
final_prices<-read_csv("crypto_simplified_final_
    prices_100.csv")


final_prices<-final_prices[-1]

final_prices <- na.locf(final_prices, na.rm =
    FALSE) # Possibly missing values present are
    replaced by the most recent non-NA prior to
    it
colSums(is.na(final_prices))


log_final_prices<-log(final_prices)
```

Main Script xxii

```
log_ret_final_prices<-as.data.frame(diff(as.
    matrix(log_final_prices)))

results_50 <- run_stock_watson_forecast(log_ret_
    final_prices)
print(results_50$Relative_MSE)
print(results_50$AR_parameter_history)
#print(results_50$Trapani_factor_number_history)

# The results are analogous to the previous one
    obtained with the smaller dataset.
# Now we ask whether a factor structure does
    actually exist in the data or not.
```

Main Script xxiii

```
# To do so it has been implemented the Trapani
   test which uses hyphotesis testing
# to check if the K largest eigenvalues of the
   sample covariance matrix are actually
# diverging to infinity.

print(results$Trapani_factor_number_history)

# We can reject with 99 % confidence, at each
   step, the null hypthotesis. Thus
# we have a statistically strong evidence that
   an estimated factor model with this data
```

```
# is not able to discriminate the signal from
    the noise. There no exists a common factor
    structure in the data.


print(results_50$Trapani_factor_number_history)

# Here in the majority of the time windows the
    test judges as highly significant just one
    common factor.
# So with a fatter dataset a common factor
    structure starts to appear in the data.
```

Main Script xxv

```
# Unfortunately this is not enaugh to choose an
    estimated factor model instead of a simple AR
     model.

# In the following part of the script a slightly
     different version of the stock_watson
    simulation is used.
# The number of eigenvalues used in estimating
    the common factors is now decided by the
    Trpani test.
# We cannot appreciate any difference in terms
    of relative MSE since also the Bai and Ng IC
    return the same number of factors.
```

Main Script xxvi

```
# Please note that in this case k_max according
    to the Schwert's rule is 5. But both
    information criteria
# and Trapani test agree on k_hat = 1.

source("real-time_forecasting2.R")

results_50_2 <- run_stock_watson_forecast(log_
    ret_final_prices)
print(results_50_2$Relative_MSE)
print(results_50_2$AR_parameter_history)
print(results_50_2$Trapani_factor_number_history
    )
```

Main Script xxvii

```
print(results$Relative_MSE)
print(results_50$Relative_MSE)
```

Real-Time Simulation Script i

```
# This algorithm implements the methodology as
    seen in
# "Stock, J. H., & Watson, M. W. (2002).
    Forecasting Using Principal Components From a
     Large Number of Predictors".
# Initially, we split our dataset in two:
# i) training dataset, composed by the 75% of
    time observations;
# ii) test dataset, composed by the remaining
    dataset.
# We are going to estimate a factor model using
    the training dataset and
# then we use it to forecast the first
    observation in the test set.
```

Real-Time Simulation Script ii

```
# In particular the common factors are estimated
    through PCA, then for each
# asset is estimated, through OLS, a linear
   model used then for the forecasting.
# Subsequently we do the same thing with the
   training dataset now containing
# also the observation we predicted in the last
   iteration.
# We keep on doing it until we have as training
   set all the initial dataset
# minus the last observation that will
   eventually predicted as last.
# In the end we compute the MSE with all the
   forecast we stored along the
```

Real-Time Simulation Script iii

```
# simulation and we compare it with the MSE of
    an AR model.
# This AR model was estimated in each iteration
    together with the factor
# model and serves as benchmark. Additionally
    the AR model is selected
# considering always the best number of
    parameters according to the Bayes
# information criterion.
# More information is present as in-line comment
     in the following code.

# Please notice that the significance of the
    estimated number of factors
```

Real-Time Simulation Script iv

```
# is tested under Trapani test as described in
    the paper
# (Trapani, L. (2018). A Randomized Sequential
    Procedure to
# Determine the Number of Factors).

# --- INITIAL SETUP ---
# data_log_returns: T x N matrix of raw log
    returns
# h: forecast horizon (e.g., 1)
# p_max: max lags for AR (e.g., 7)

run_stock_watson_forecast <- function(data_log_
    returns, h = 1, p_max = 7) {
```

Real-Time Simulation Script v

```
library(dfms)  # For ICr
library(HDRFA) # For PCA
source("trapani_factor_test.R")

T_total <- nrow(data_log_returns)
N_assets <- ncol(data_log_returns)
T1 <- floor(0.75 * T_total)
#browser()
k_max <- floor(8 * (min(N_assets, T_total) /
    100)^(1/4))
# Using Schwert's rule to compute the upper
    limit of the number of common factors on
    which V(k) is computed when using ICr
```

Real-Time Simulation Script vi

```
# Containers for results
n_forecasts = T_total - (T1 + h) + 1 # it
    coincides with the number of iterations of
    the outer-most for loop: at iteration t, we
     forecast y_t+h starting from t=T1
# In actuals[1, i] there will be y_(T1+h)_i, i
     is the cross-sectional unit
actuals <- matrix(NA, nrow = n_forecasts, ncol
     = N_assets)
forecasts_factor <- matrix(NA, nrow = n_
    forecasts, ncol = N_assets)
forecasts_ar <- matrix(NA, nrow = n_forecasts,
     ncol = N_assets)
```

Real-Time Simulation Script vii

```
p_history <- matrix(NA, nrow = n_forecasts,
    ncol = N_assets)
k_trapani_history <- rep(NA, n_forecasts)

row_idx <- 1

mse_f <- numeric(N_assets)
mse_ar <- numeric(N_assets)
rel_mse <- numeric(N_assets)

u_vec <- c(sqrt(2), -sqrt(2)) # u1 and u2
    exctracted from the distribution F(u)
weights_vec <- c(0.5, 0.5) # F(u1) and F(u2),
    as prescribed by Trapani paper
```

Real-Time Simulation Script viii

```
# Real-time forecasting loop:
# for each t we compute a forecast assuming
   not to have knowledge about the future (
   data)
for (t in T1:(T_total - h)) {

  # We assume to have data up to index 't', we
      want to forecast y_t+h
  D_train_t <- data_log_returns[1:t, ]

  # In order to properly estimate the factors
     through PCA we need to standardize our
     data
```

Real-Time Simulation Script ix

```
# We calculate mean/SD from training to
    avoid look-ahead bias
train_means <- colMeans(D_train_t)
train_sds <- apply(D_train_t, 2, sd) # Apply
    to all the cols the function sd from the
    package stats
D_std_t <- scale(D_train_t, center = train_
    means, scale = train_sds)

# We estimate the number of factors applying
    eigenvalue thresholding techinque
ic <- ICr(D_std_t, k_max) # This function
    return k_hat as the minimizer of 3
    information criteria (IC)
```

Real-Time Simulation Script x

```
# proposed by Bai and Ng (2002)
# In general all of them overestimate the
    estimated number of common factors k_hat.
estimated_factor_terms <- PCA(as.matrix(D_
    std_t), min(ic$r.star), constraint = "L")
     # We finally estimate the common factors
    .
#browser()
F_hat <- estimated_factor_terms$Fhat
#F_hat <- as.data.frame(F_hat)
#browser()

pred_f <- numeric(N_assets)
best_pred_ar <- numeric(N_assets)
```

Real-Time Simulation Script xi

```
k_trapani_history[row_idx] <- trapani_factor
    _test(ic$eigenvalues, N_assets, t, u_vec,
     weights_vec, alpha = 0.01)
# See Trapani, L. (2018). A Randomized
    Sequential Procedure to Determine the
    Number of Factors

for (i in 1:N_assets) {
  # We use OLS estimator to fit a linear
      model in order to forecast y_t+h
  y_raw <- D_train_t[(1 + h):t, i]
```

Real-Time Simulation Script xii

```
X_factors <- as.data.frame(F_hat[1:(t - h)
    , , drop = FALSE]) # "drop = FALSE"
    ensures that each row is treated as a
    data frame object instead of 'vector'
#browser()
model_factor <- lm(y_raw ~., data = X_
    factors) # A typical model has the form
     response ~ terms
# y_raw has to be a numeric vector

latest_F <- as.data.frame(matrix(F_hat[t,
    ], nrow = 1)) # matrix creates a matrix
     from the given set of values
colnames(latest_F) <- colnames(X_factors)
```

Real-Time Simulation Script xiii

```
pred_f[i] <- predict(model_factor, newdata
    = latest_F) #y_t+h = beta_OLS * F_hat_
   t

# We are going to compare our estimated
   factor model with an autoregressive
   model (AR)
best_bic <- Inf
best_p <- 0
# We use Bayesian Information Criterion (
   BIC) to select the number of parameters
    of the AR model at time t
# Fixed window start for fair BIC
   comparison
```

Real-Time Simulation Script xiv

```
obs_start <- p_max + h

for (p in 0:p_max) {

  if (p == 0) {
      ar_mod <- lm(D_train_t[(1 + h):t, i]
          ~ 1)
  } else {
    # Create lag matrix
    lags_matrix <- embed(D_train_t[1:t, i
       ], p + 1) # If p = 0 it just
       returns D_train_t[1:t, 1]
```

```
# Read the documentation at https://
    stat.ethz.ch/R-manual/R-devel/
    library/stats/html/embed.html
# It is as if the headers of the p+1
    columns were: lag_0,lag1, ..., lag_
    p-1, lag_p
# For instance, considering AR(p), y_t
    +h+p will be regressed on the t-th
    row of lags_matrix considering cols
     from the second, corresponding to
    lag_1, to the last one
y_shifted <- D_train_t[(p + h):t, i]
```

Real-Time Simulation Script xvi

```
x_shifted <- as.data.frame(lags_matrix
    [1:(nrow(lags_matrix) - h + 1), 2:(
    p + 1), drop = FALSE])
# Please note that the last y to be
    predicted is y_t, which is
    regressed on y_t-h, y_t-h-1, ..., y
    _t-h-p+1
# For this reason the last row to be
    included in x_shifted is the t-h+1^
    th because from the second column
    we
# find the regressors required
```

Real-Time Simulation Script xvii

```
        ar_mod <- lm(y_shifted ~., data = x_
            shifted)
      }

      # Calculate BIC on the slice [p_max to t
          -h]
      # We consider this slice to get a
          comparable RSS (same observations)
      slice_y <- D_train_t[obs_start:t, i]
      resids <- residuals(ar_mod)
      # We take the last n_obs values of
          resids and compute RSS with them
      n_obs <- length(slice_y)
      rss <- sum(tail(resids, n_obs)^2)
```

Real-Time Simulation Script xviii

```
        current_bic <- n_obs * log(rss / n_obs)
            + (p + 1) * log(n_obs) #
            Theoretically should be p+2 in order
            to count also the variance of AR
            irreducible error

        if (current_bic < best_bic) {
          best_bic <- current_bic
          best_p <- p

          # Generate the forecast for this 'p'
          if (p == 0) {
```

```
            best_pred_ar[i] <- predict(ar_mod,
                newdata = data.frame(1))[1] #
                data.frame(1) is just a 1 given
                to the model to let it predict
                the intercept
        } else {
            new_lags <- as.data.frame(matrix(D_
                train_t[(t - p + 1):t, i], nrow =
                 1)) # Here we are transposing a
                column in a row
            new_lags <- new_lags[, rev(seq_len(p
                )), drop = FALSE]
```

Real-Time Simulation Script xx

```
            # seq_len(p) generates [1, 2, ..., p
              ], with rev() we reverse it, then
               we rearrange the order to
              imitate the order of lags_matrix
              cols (newest to oldest)
            colnames(new_lags) <- colnames(x_
              shifted)
            best_pred_ar[i] <- predict(ar_mod,
              newdata = new_lags)
        }
    }
}

# We append the results computed at time t
```

Real-Time Simulation Script xxi

```
      actuals[row_idx, i] <- data_log_returns[t
         + h, i]
      forecasts_factor[row_idx, i] <- pred_f[i]
      forecasts_ar[row_idx, i] <- best_pred_ar[i
         ]
      p_history[row_idx, i] <- best_p

    }
    row_idx <- row_idx + 1
  }

  # Finally we compare the factor model with the
      AR model using relative MSE
  #browser()
```

Real-Time Simulation Script xxii

```
mse_f <- colMeans((actuals - forecasts_factor)
    ^2)
mse_ar <- colMeans((actuals - forecasts_ar)^2)
rel_mse <- mse_f / mse_ar

return(list(
  Relative_MSE = rel_mse,
  Factor_MSE = mse_f,
  AR_MSE = mse_ar,
  Actuals = actuals,
  Forecasts_Factor = forecasts_factor,
  AR_parameter_history = p_history,
  Trapani_factor_number_history = k_trapani_
      history
```

Real-Time Simulation Script xxiii

```
  ))
}

# Example usage:
# results <- run_stock_watson_forecast(my_
   financial_data)
# print(results$Relative_MSE)
```

Trapani Test Script i

```
# A test to check if the larger k eigenvalues
   diverge to infinity using a randomised
# test statistic based directly on the estimated
    eigenvalues.
# Details are provided in:
# Trapani, L. (2018). A Randomized Sequential
   Procedure to Determine the Number of Factors.
# https://doi.org/10.1080/01621459.2017.1328359

trapani_factor_test <- function(estimated_
   eigenvalues, N, T_total, u_vec, weights_vec,
   alpha = 0.05, R = 400) {
  # N = cross-sectional sample size, T_total =
     time sample size
```

Trapani Test Script ii

```
# estimated_eigenvalues = vector of
    eigenvalues
# u_vec contains the value of u drawn by a
    distribution F(u)
# weights_vec contains the values of F(u)

if (length(u_vec) != length(weights_vec)) {
  return(-1)
}

# The method requires the eigenvalues to be
    ordered in ascending order
eigenvalues <- sort(estimated_eigenvalues,
    decreasing = TRUE)
```

Trapani Test Script iii

```
beta <- log(N) / log(T_total)

if (beta <= 0.5) {
  delta <- 0.01
} else {
  delta <- 1.01 * (1 - 1 / (2 * beta))
}

if (N <= T_total) {
  avg_eigs = mean(eigenvalues)
  phi <- exp(N^(-delta) * (estimated_
      eigenvalues / avg_eigs))
}
```

Trapani Test Script iv

```
estimated_k <- 0


for (p in 0:((length(estimated_eigenvalues))
    -1)) {
  if (N > T_total) {
    eigs_sum = sum(eigenvalues[(p+1):length(
        eigenvalues)])
    avg_eigs = eigs_sum / N
    phi_p <- exp(N^(-delta) * (estimated_
        eigenvalues[p+1] / avg_eigs))
  } else {
    phi_p = phi[p+1]
```

```
}
# Generate artificial sample of R
    observations xi ~ i.i.d. N(0,1)
xi <- rnorm(R, mean = 0, sd = 1)

#
theta_squared_values <- sapply(u_vec,
    function(u) {

  zeta_seq <- as.numeric(sqrt(phi_p) * xi <=
      u)

  theta <- (2 / sqrt(R)) * sum(zeta_seq -
      0.5)
```

Trapani Test Script vi

```
    return(theta^2)
  })

  test_statistic <- sum(theta_squared_values *
      weights_vec)

  # Under H0, test_statistic follows a chi-
     squared distribution with df=1
  critical_value <- qchisq(1 - alpha, df = 1)

  if (test_statistic > critical_value) {
    # Reject H0: The eigenvalue does not
       diverge
```

Trapani Test Script vii

```
      break
    } else {
      # Fail to reject H0: The eigenvalue
          diverges (a significant factor)
      estimated_k <- p+1
    }
  }

  return(estimated_k)
}
```